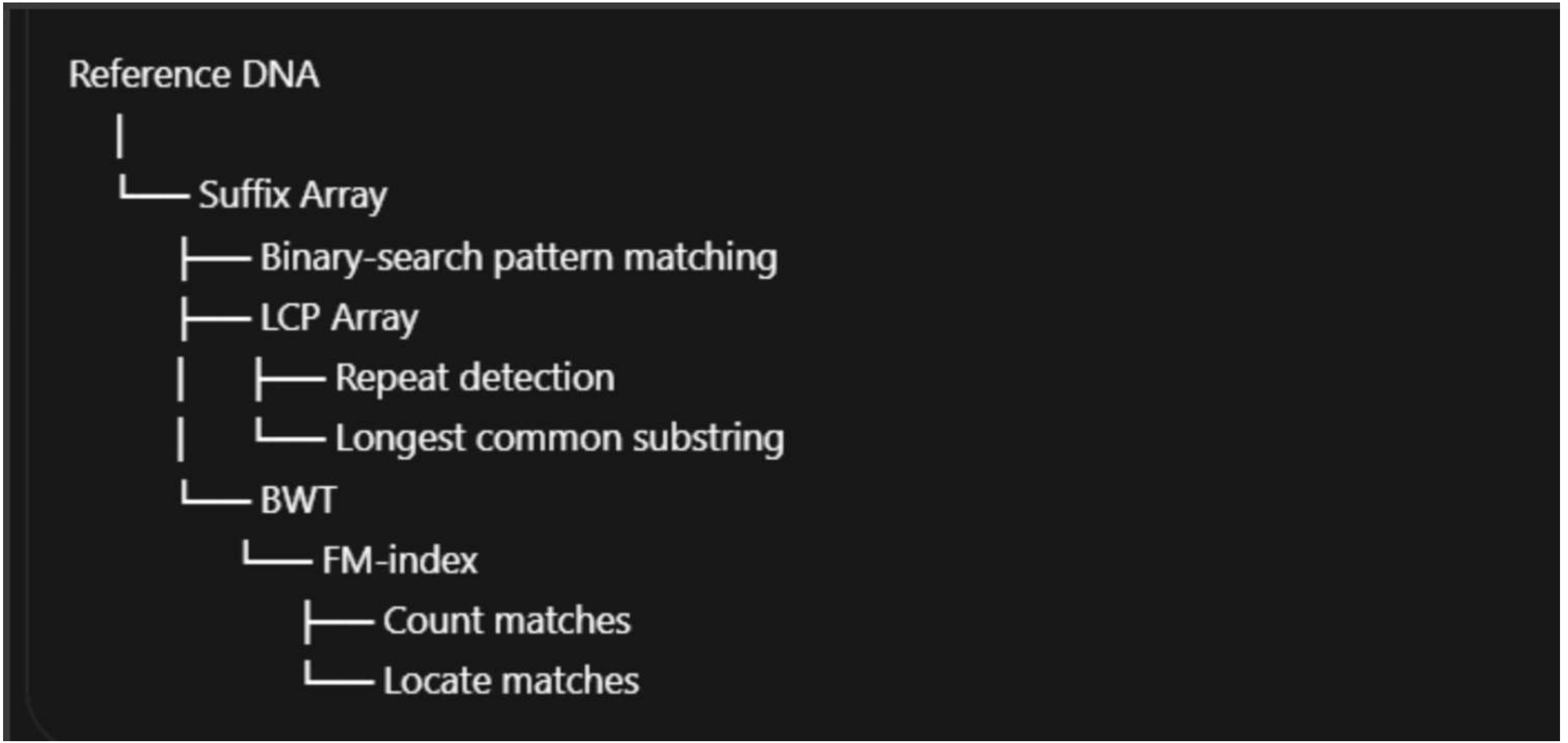
Suffix arrays, BWT and FM-index

Two complete worked examples

ACAC / query AC
GATAGA / query GA


Reference DNA
Suffix Array
Binary-search pattern matching
LCP Array
Repeat detection
Longest common substring
BWT
FM-index
Count matches
Locate matches

The structures and the notation

One reference supports several kinds of analysis

Structure or symbol	Meaning
Suffix array (SA)	Starting positions of suffixes in alphabetical order
BWT	The character before each sorted suffix
C[x]	Number of reference characters smaller than x
Occ(x, i)	Count of x in BWT positions 0 through i − 1
[lo, hi)	Rows lo through hi − 1. Count = hi − lo.
$	Unique end marker. It sorts before A, C, G and T.

Example 1: suffix-array construction

Reference ACAC. Query AC.

Original position	Suffix before sorting	Sorted suffix	SA position
0	ACAC$	$	4
1	CAC$	AC$	2
2	AC$	ACAC$	0
3	C$	C$	3
4	$	CAC$	1

SA = [4, 2, 0, 3, 1]

Example 1: BWT construction

Take the character immediately before each suffix in the original reference

Row	Sorted suffix	SA position	Preceding character
0	$	4	C
1	AC$	2	C
2	ACAC$	0	$
3	C$	3	A
4	CAC$	1	A

ACAC$ becomes BWT = CC$AA

Example 1: C dictionary

Sort the present alphabet as $, A, C

Character	Frequency	Earlier character counts	C value
$	1	0	0
A	2	1	1
C	2	1 + 2	3

Save the running total before adding the current character’s frequency.

Sorted characters: $ A A C C

Example 1: Occ counts

BWT = CC$AA. The boundary i excludes position i.

Boundary i	BWT prefix before i	Occ(A, i)	Occ(C, i)
0	empty	0	0
1	C	0	1
2	CC	0	2
3	CC$	0	2
4	CC$A	1	2
5	CC$AA	2	2

Example 1: first backward-search step

Query AC. Process the last character C first.

Candidate row	Suffix beginning with C
3	C$
4	CAC$

Initial range [0, 5) contains every suffix row.

C[C] = 3. Occ(C, 0) = 0. Occ(C, 5) = 2.

new lo = 3 + 0 = 3
new hi = 3 + 2 = 5
New range = [3, 5)

Example 1: second backward-search step

Extend C to AC by processing A

Matching row	Matching suffix	Reference position
1	AC$	2
2	ACAC$	0

Old range [3, 5). C[A] = 1.

Occ(A, 3) = 0. Occ(A, 5) = 2.

new lo = 1 + 0 = 1
new hi = 1 + 2 = 3
Final range = [1, 3)

Example 1: recovering positions

With sampling interval 16, this tiny reference stores only position 0

Current row	BWT character	LF calculation	Next row
1	C	3 + Occ(C,1) = 3 + 1	4
4	A	1 + Occ(A,4) = 1 + 1	2

Stored sample: row 2 has reference position 0.

Row 1 takes 2 LF steps to reach that sample.

Row 1: position = 0 + 2 = 2
Row 2: stored position = 0
Sorted locations = [0, 2]

Example 1: inverse BWT

LF-mapping can also reconstruct the original reference

Step	Current row	Character read from BWT	Next row
1	0	C	3
2	3	A	1
3	1	C	4
4	4	A	2

Collected backward: C A C A
Reverse and append $: A C A C $

Example 2: suffix-array construction

Reference GATAGA. Query GA.

Original position	Suffix before sorting	Sorted suffix	SA position
0	GATAGA$	$	6
1	ATAGA$	A$	5
2	TAGA$	AGA$	3
3	AGA$	ATAGA$	1
4	GA$	GA$	4
5	A$	GATAGA$	0
6	$	TAGA$	2

SA = [6, 5, 3, 1, 4, 0, 2]

Example 2: BWT construction

Read the preceding-character column in suffix-array order

Row	Sorted suffix	SA position	BWT character
0	$	6	A
1	A$	5	G
2	AGA$	3	T
3	ATAGA$	1	G
4	GA$	4	A
5	GATAGA$	0	$
6	TAGA$	2	A

BWT = A G T G A $ A

Example 2: C dictionary calculation

Present alphabet: $, A, G, T. C is absent from this reference.

Char.	Frequency	Total before	C value	Total after
$	1	0	0	1
A	3	1	1	4
G	2	4	4	6
T	1	6	6	7

C[G] = 4 because $ and three As sort before G.

Sorted characters: $ A A A G G T

Example 2: Occ counts

BWT = AGTGA$A. Count only characters before boundary i.

i	BWT prefix before i	Occ(A, i)	Occ(G, i)	Occ(T, i)
0	empty	0	0	0
1	A	1	0	0
2	AG	1	1	0
3	AGT	1	1	1
4	AGTG	1	2	1
5	AGTGA	2	2	1
6	AGTGA$	2	2	1
7	AGTGA$A	3	2	1

Example 2: first backward-search step

Query GA. Process A first.

Candidate row	Suffix beginning with A
1	A$
2	AGA$
3	ATAGA$

Initial range [0, 7) contains every suffix row.

C[A] = 1. Occ(A, 0) = 0. Occ(A, 7) = 3.

new lo = 1 + 0 = 1
new hi = 1 + 3 = 4
New range = [1, 4)

Example 2: why one candidate disappears

To extend A to GA, check the recorded preceding character

Old row	A-starting suffix	BWT character	Extension
1	A$	G	GA$ survives
2	AGA$	T	TAGA$ does not match GA
3	ATAGA$	G	GATAGA$ survives

The BWT already records the preceding character.

The search uses counts in this interval instead of checking each suffix string.

Example 2: second backward-search step

Extend A to GA by processing G

Matching row	Matching suffix	Reference position
4	GA$	4
5	GATAGA$	0

Old range [1, 4). C[G] = 4.

Occ(G, 1) = 0. Occ(G, 4) = 2.

new lo = 4 + 0 = 4
new hi = 4 + 2 = 6
Final range = [4, 6)

Example 2: position recovery with LF

Stored sample: row 5 has reference position 0

Step	Current row	BWT	LF calculation	Next row
1	4	A	1 + Occ(A,4) = 1 + 1	2
2	2	T	6 + Occ(T,2) = 6 + 0	6
3	6	A	1 + Occ(A,6) = 1 + 2	3
4	3	G	4 + Occ(G,3) = 4 + 1	5

Row 4: 0 + 4 LF steps = 4
Row 5: stored position = 0
Sorted locations = [0, 4]

Example 2: inverse BWT

Start at row 0 and collect six characters backward

Step	Current row	Read BWT character	Next row
1	0	A	1
2	1	G	4
3	4	A	2
4	2	T	6
5	6	A	3
6	3	G	5

Collected: A G A T A G
Reverse and append $: G A T A G A $

The computer’s index-building method

The implementation prepares reusable search information

```
t = reference + "$"
sa = build_suffix_array(t)
bwt = "".join(t[pos - 1] for pos in sa)

C = {}
total = 0
for c in sorted(set(t)):
    C[c] = total
    total += t.count(c)
```

Then save Occ checkpoints every B characters.

Store (row, position) when position % s == 0.

The computer’s backward-search method

Both new boundaries use the same old interval

```
lo, hi = 0, n
for c in reversed(pattern):
    if c not in C:
        return "no matches"
    new_lo = C[c] + Occ(c, lo)
    new_hi = C[c] + Occ(c, hi)
    lo, hi = new_lo, new_hi
    if lo >= hi:
        return "no matches"
return lo, hi
```

Count = hi − lo. Matching rows are lo through hi − 1.

Checkpoints and sampled positions

The index stores selected information and recovers the rest

Operation	Computer method	Default
Occ(c, i)	Use the saved count at the earlier boundary, then count c in the remaining BWT section.	B = 64
Recover a position	Follow LF until a sampled row. Add the number of steps to its stored position.	s = 16

Occ(c, i) = checkpoint[c][i // B]
+ count(c in BWT[(i // B) * B : i])

LF(row) = C[BWT[row]] + Occ(BWT[row], row)

COMPLEXITY

Time complexity in this project

n: reference length including $. m: query length. occ: match count.

Operation	Time complexity	Reason
Build suffix array	$O(n \log^2 n)$	Prefix doubling with comparison sorting
Build FM-index with an existing SA	O(n), fixed DNA alphabet	BWT, character counts, checkpoints, sampling
One Occ query	O(B)	Scan at most one checkpoint block
Count matches	O(mB)	Backward search over m query characters
Locate and sort all matches	$O(mB + occ \cdot s \cdot B + occ \log occ)$	Search, LF recovery for each hit, sorting

B is the Occ checkpoint interval. s is the SA sampling interval.

Overall index construction includes the suffix-array cost.

Both examples: rows and positions

Matching intervals identify table rows. Locating returns DNA coordinates.

Result	Example 1	Example 2
Reference	ACAC	GATAGA
Query	AC	GA
Suffix array	[4, 2, 0, 3, 1]	[6, 5, 3, 1, 4, 0, 2]
BWT	CC$AA	AGTGA$A
Final interval	[1, 3)	[4, 6)
Matching rows	1 and 2	4 and 5
Match count	2	2
Sorted reference positions	[0, 2]	[0, 4]